T3000 API & Protocol

Message 15: LOGGING_DATA · built Nov 13, 2026

Message 15: LOGGING_DATA

The LOGGING_DATA message (Action 15) triggers comprehensive data collection from all panel points. It reads inputs, outputs, and variables for logging purposes, similar to GET_WEBVIEW_LIST but for bulk data capture.

Important: This feature has a 1-minute cooldown and is disabled by default in release builds.

Overview

Action: `LOGGING_DATA` (15) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 3040
Purpose: Bulk data collection for all panel points **Status:** ⚠ Disabled by default in release builds

Request Format

JSON Structure:

JSON

```
{
  "action": "LOGGING_DATA",
  "panelId": 0,
  "serialNumber": 237219
}
```

Request Parameters

Parameter	Type	Required	Description
`panelId`	number	Yes	Target panel ID
`serialNumber`	number	Yes	Panel serial number

Response Format

JSON Structure:

JSON

```
{
  "action": "LOGGING_DATA_RES",
  "data": [
    {
      "panel_id": 0,
      "panel_name": "Main Panel",
      "panel_serial_number": 237219,
      "panel_ipaddress": "192.168.1.100",
      "input_logging_time": "2026-01-08 10:30:00",
      "output_logging_time": "2026-01-08 10:30:05",
      "variable_logging_time": "2026-01-08 10:30:10",
      "inputs": [...],
      "outputs": [...],
      "variables": [...]
    }
  ]
}
```

Implementation

Location: BacnetWebView.cpp line 3040

C++

```cpp
case LOGGING_DATA:
{
    // Disabled in release builds
    if (enable_trendlog_background_read == false)
    {
        break;
    }

    int temp_panel_id = json.get("panelId", Json::nullValue).asInt();
    int temp_serial_number = json.get("serialNumber", Json::nullValue).asInt();

    // Cooldown: 1 minute between requests
    static DWORD last_logging_time = 0;
    DWORD current_time = GetTickCount();
    if (current_time - last_logging_time < 60 * 1000)
    {
        OutputDebugString(_T("LOGGING_DATA SKIPPED - within 1 minute cooldown\n"));
        break;
    }
    last_logging_time = current_time;

    Json::Value tempjson;
    tempjson["action"] = "LOGGING_DATA_RES";

    // Trigger trendlog reading
    int nret = Post_ReadTrendlog_Message(temp_panel_id, temp_serial_number);
    if (nret < 0)
    {
        WrapErrorMessage(builder, tempjson, outmsg, _T("Panel is offline"));
        break;
    }

    // Collect data from all validated panels
    int device_count = 0;
    for (int panel_idx = 0; panel_idx < g_bacnet_panel_info.size(); panel_idx++)
    {
        int npanel_id = g_bacnet_panel_info.at(panel_idx).panel_number;

        // Validation checks
        if (g_bacnet_panel_info.at(panel_idx).object_instance != g_logging_time[npanel_i
```

```
            continue;
        if (g_bacnet_panel_info.at(panel_idx).nseiral_number != g_logging_time[npanel_id
            continue;
        if (g_bacnet_panel_info.at(panel_idx).panel_number != g_logging_time[npanel_id].
            continue;
        if (g_logging_time[npanel_id].basic_setting_status != 1)
            continue;

        // Get IP address
        unsigned char* ipAddr = g_Device_Basic_Setting[npanel_id].reg.ip_addr;
        char ipStr[16];
        sprintf(ipStr, "%d.%d.%d.%d", ipAddr[0], ipAddr[1], ipAddr[2], ipAddr[3]);

        // Add device info
        tempjson["data"][device_count]["panel_id"] = npanel_id;
        tempjson["data"][device_count]["panel_name"] = (char*)g_Device_Basic_Setting[npa
        tempjson["data"][device_count]["panel_serial_number"] = g_Device_Basic_Setting[r
        tempjson["data"][device_count]["panel_ipaddress"] = ipStr;
        tempjson["data"][device_count]["input_logging_time"] = g_logging_time[npanel_id]
        tempjson["data"][device_count]["output_logging_time"] = g_logging_time[npanel_id
        tempjson["data"][device_count]["variable_logging_time"] = g_logging_time[npanel_

        // Add inputs, outputs, variables data...
        device_count++;
    }

    const std::string output = Json::writeString(builder, tempjson);
    outmsg = CString(output.c_str());
}
```

Behavior

Cooldown Period

1-minute cooldown between requests to prevent server overload. Subsequent requests within 60 seconds are silently ignored.

Release Build Disable

Controlled by `enable_trendlog_background_read` flag:

- **Debug builds:** Typically enabled

- **Release builds:** Disabled by default

Data Collection

Collects from all validated panels:

- Panel basic info (name, serial, IP)
- Last logging timestamps
- All inputs (0-63)
- All outputs (0-63)
- All variables (0-63)

Frontend Usage

TYPESCRIPT

```typescript
const requestLoggingData = async (panelId: number, serialNumber: number) => {
  const message = {
    action: 'LOGGING_DATA',
    panelId: panelId,
    serialNumber: serialNumber
  };

  sendMessage(JSON.stringify(message));

  return new Promise((resolve, reject) => {
    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'LOGGING_DATA_RES') {
        if (response.error) {
          reject(new Error(response.error));
        } else {
          resolve(response.data);
        }
        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);

    setTimeout(() => {
      webSocket.off('message', handler);
      reject(new Error('Logging timeout'));
    }, 30000);  // 30 second timeout
  });
};
```

With Cooldown Management

TYPESCRIPT

```typescript
class LoggingManager {
  private lastRequest: number = 0;
  private readonly COOLDOWN_MS = 60 * 1000;  // 1 minute

  async requestLogging(panelId: number, serialNumber: number) {
    const now = Date.now();
    const timeSinceLastRequest = now - this.lastRequest;

    if (timeSinceLastRequest < this.COOLDOWN_MS) {
      const remaining = this.COOLDOWN_MS - timeSinceLastRequest;
      throw new Error(`Cooldown active. Wait ${Math.ceil(remaining / 1000)}s`);
    }

    this.lastRequest = now;
    return await requestLoggingData(panelId, serialNumber);
  }
}

const logger = new LoggingManager();

// Usage
try {
  const data = await logger.requestLogging(0, 237219);
  console.log('Logging data:', data);
} catch (error) {
  console.error('Logging failed:', error);
}
```

Comparison: Action 15 vs Action 17

Feature	LOGGING_DATA (15)	GET_WEBVIEW_LIST (17)
Purpose	Bulk logging	Selective retrieval
Cooldown	1 minute	None
Data Scope	All panels, all points	Specific range
Release Build	Disabled	Enabled
Use Case	Background logging	UI data display

Recommendation: Use GET_WEBVIEW_LIST (17) for regular data retrieval. Reserve LOGGING_DATA for periodic bulk snapshots.

Limitations

Disabled by Default

Not available in standard release builds. Enable by setting `enable_trendlog_background_read = true`.

Rate Limiting

1-minute cooldown prevents frequent polling. For real-time updates, use GET_WEBVIEW_LIST instead.

Panel Validation

Only includes panels that pass strict validation:

- Instance match
- Serial number match
- Panel number match
- Basic settings loaded

See Also

- GET_WEBVIEW_LIST - Selective data retrieval (Action 17)
- GET_PANEL_DATA - Cached panel data
- Platform Overview - Architecture